Juice Shop Performance Test Report

OWASP Juice Shop v20.3.0 · k6 v2.3.0 · 27 October 2026

1. Summary

All three modules pass their load tests at the planned load (50 concurrent shoppers for search, 30 for login and basket): no failed requests, every check passed, and every request type well inside its response-time limit.

Product search breaks between 40 and 50 requests per second. Up to 40 req/s, the 95th percentile stays under 20 ms. At 50 req/s it jumps to 907 ms, and at 100 req/s to 17.8 s. The cause is CPU: Juice Shop runs as a single Node.js process, and at about 45 req/s of search its main thread is fully busy, so new requests queue.

Past that point, Juice Shop doesn't recover cleanly. Memory grows from about 250 MB to 2.5-3 GB while requests queue, and stays there after the load stops. A single search then takes about 80-130 ms instead of 42-48 ms until the process is restarted.

One functional defect also turned up: adding a product that is already in the basket returns **HTTP 500** instead of a 4xx error.

Test	Peak users	Requests	Throughput	p95 (all)	Failed requests	Checks passed	Result
Product Search	50	5,572	20.5/s	58 ms	0.00%	100.00%	✓ Pass
Login	30	6,177	22.8/s	71 ms	0.00%	100.00%	✓ Pass
Basket	30	5,542	20.0/s	116 ms	0.00%	100.00%	✓ Pass
Stress (search)	up to 1,000	20,119	10-100/s target	Meets target up to 40 req/s; fails from 50 req/s			✗ Breaking point found

2. Scope and approach

Application	OWASP Juice Shop v20.3.0: an open-source Angular + Node.js (Express) e-commerce app with a SQLite database. Default configuration, one Node.js process.
Modules	**Product Search** (catalogue search and product details, anonymous), **Login** (authentication and profile), **Basket** (view, add, change quantity, remove; logged in). Each is tested on its own so results can be attributed to it.
Tool	k6 v2.3.0. Scripts are JavaScript, pass/fail limits are thresholds that set the exit code, and the arrival-rate executor makes a fair stress test. See `docs/tool-selection.md`.
Test types	**Smoke**: 1 user, 1 pass, to prove the scripts work. **Load**: per module, ramp up 1 min, hold 3 min, ramp down 30 s. **Stress**: search at a rising fixed request rate, 1 min per step, to find the breaking point.
Realistic traffic	Think time of 1-3 s between actions, a mix of request types per module (for example 70% searches that find products, 10% that find nothing, 20% product pages), varied search terms, and one account per virtual user so sessions and baskets are never shared. Test accounts are created in `setup()`, before the load starts, so registration isn't measured.
Assertions	Every response is checked for content, not only status (a search returns products, the basket contains the added item, a wrong password gets 401). Thresholds per request type: p95 < 500 ms for reads, < 800 ms for login and basket writes, < 1% failed requests, > 99% checks passed.
Environment	Laptop with an Intel Core i5-1235U (10 cores / 12 threads), 39.7 GB RAM and an NVMe SSD, on Windows 11 Pro. Node.js v24.21.0. k6 and Juice Shop on the same machine, over loopback (no network latency).
Monitoring	k6 records response time, throughput, errors and checks, with an HTML dashboard per test. `run.ps1` samples Juice Shop's CPU (100% = one full core) and memory every 2 s.

3. Load test results by module

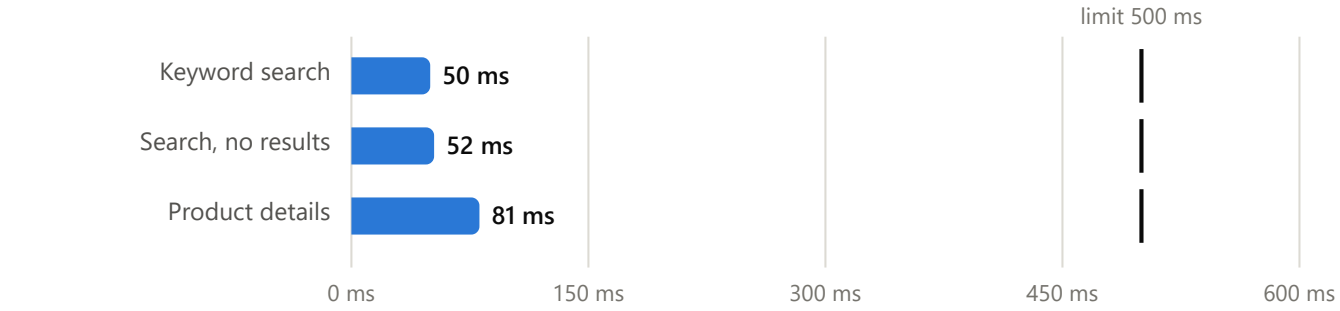
Product Search

5,572 requests **20.5/s** average throughput **0.00%** failed **100.00%** checks passed (11,142)

52% / 84% server CPU avg / peak **265 MB** peak server memory

Request	Share	Avg	Median	p90	p95	Max	Limit (p95)	Result
Keyword search	70%	28 ms	13 ms	38 ms	50 ms	1.43 s	< 500 ms	✓ Pass
Search, no results	10%	30 ms	14 ms	39 ms	52 ms	1.36 s	< 500 ms	✓ Pass
Product details	20%	54 ms	30 ms	60 ms	81 ms	2.04 s	< 500 ms	✓ Pass

Product Search: 95th-percentile response time by request



Bars show the 95th-percentile response time; the vertical tick on each row marks that request's pass/fail limit.

Passes with a wide margin. Every request type is at least 6x inside its 500 ms limit, with no failed requests across 5,572 requests at 50 concurrent users. All 11,142 checks passed, so every search returned products and every empty search returned an empty list.

- **Product details is the slowest request** (p95 81 ms, against 50 ms for a search), although it returns a single product. It is served by Juice Shop's generic REST API (`/api/Products/<id>`) rather than the hand-written search route, so the generic layer appears to cost more per request than the query itself.
- **A few outliers of 1.4-2.0 s** (the max column) don't affect the p95. They are isolated and fit a pause such as garbage collection, but this test can't confirm the cause.
- Searches that find nothing cost the same as ones that do, which is consistent with the time going on per-request overhead rather than on result size.

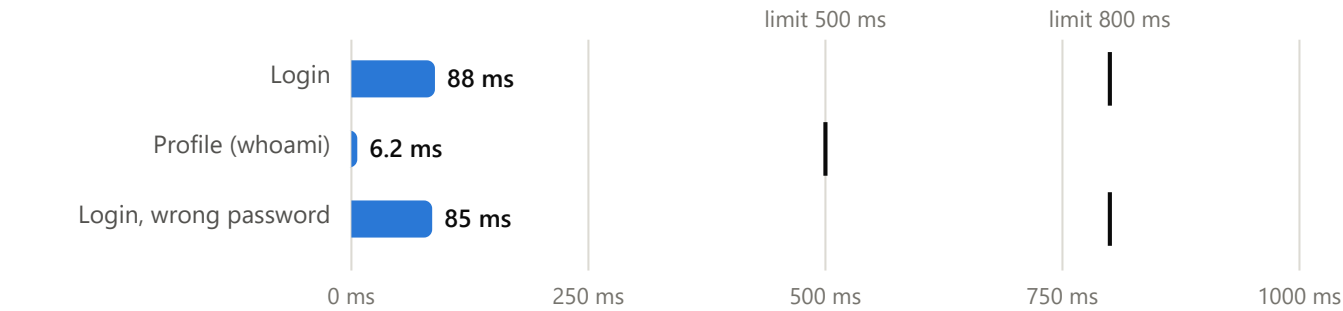
Login

6,177 requests **22.8/s** average throughput **0.00%** failed **100.00%** checks passed (9,002)

56% / 90% server CPU avg / peak **281 MB** peak server memory

Request	Share	Avg	Median	p90	p95	Max	Limit (p95)	Result
Login	85%	35 ms	26 ms	67 ms	88 ms	331 ms	< 800 ms	✓ Pass
Profile (whoami)	85%	2.7 ms	2.3 ms	4.7 ms	6.2 ms	55 ms	< 500 ms	✓ Pass
Login, wrong password	15%	29 ms	18 ms	66 ms	85 ms	227 ms	< 800 ms	✓ Pass

Login: 95th-percentile response time by request



Bars show the 95th-percentile response time; the vertical tick on each row marks that request's pass/fail limit.

Passes with a wide margin. Login p95 is 88 ms against an 800 ms limit, and the profile lookup is 6 ms. All 9,002 checks passed: every successful login returned a token, and that token identified the right user.

- **A rejected login costs as much as a successful one** (p95 85 ms against 88 ms). So most of the time goes on looking up the account, not on issuing the token.
- **No protection against password guessing.** The test made 497 wrong-password attempts, about 16 per account in 4.5 minutes. Every one was answered at full speed, with no lockout, delay or rate limit. This is a security gap rather than a performance one, but load testing is where it shows (see finding 6).

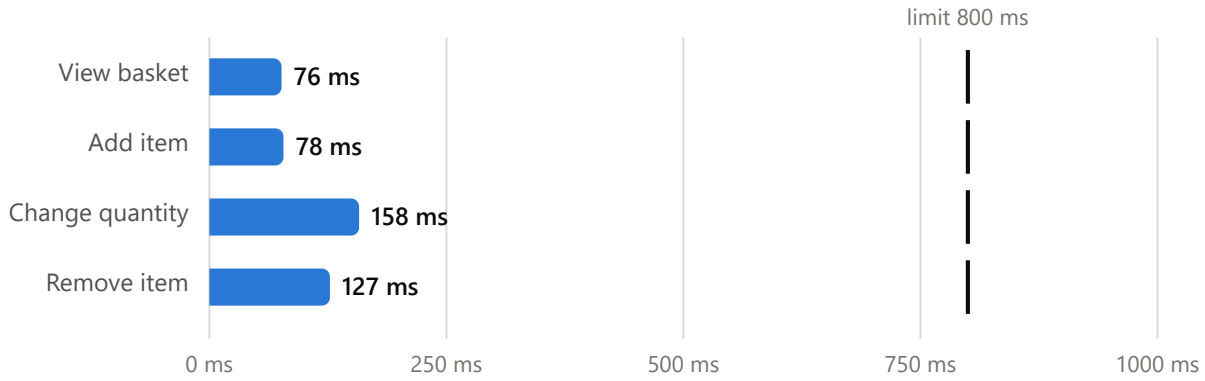
Basket

5,542 requests **20.0/s** average throughput **0.00%** failed **100.00%** checks passed (6,636)

49% / 83% server CPU avg / peak **286 MB** peak server memory

Request	Share	Avg	Median	p90	p95	Max	Limit (p95)	Result
View basket	40%	36 ms	30 ms	61 ms	76 ms	183 ms	< 800 ms	✓ Pass
Add item	20%	27 ms	18 ms	59 ms	78 ms	170 ms	< 800 ms	✓ Pass
Change quantity	20%	68 ms	52 ms	134 ms	158 ms	285 ms	< 800 ms	✓ Pass
Remove item	20%	59 ms	48 ms	104 ms	127 ms	220 ms	< 800 ms	✓ Pass

Basket: 95th-percentile response time by request



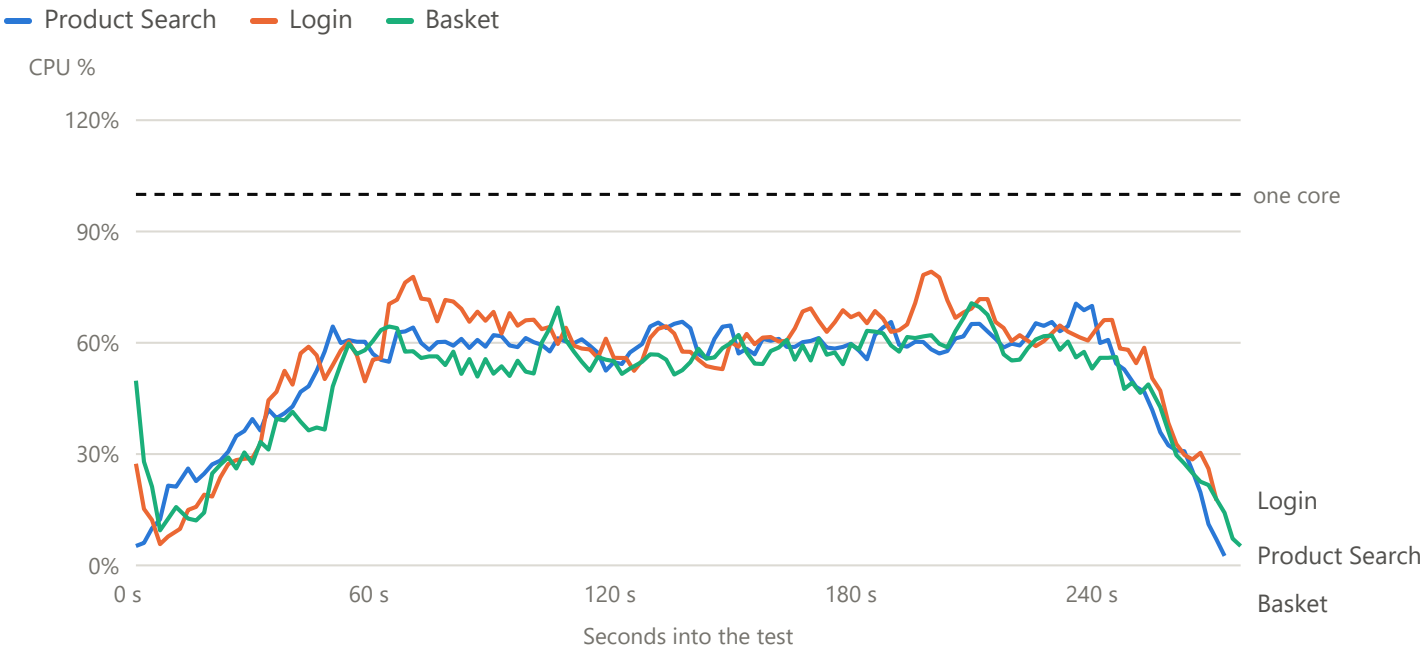
Bars show the 95th-percentile response time; the vertical tick on each row marks that request's pass/fail limit.

Passes with a wide margin. The slowest action, changing the quantity, has a p95 of 158 ms against an 800 ms limit. All 6,636 checks passed, including that each added product appears in the basket, so 30 users writing at the same time didn't corrupt each other's baskets.

- **Writes are about twice as slow as reads:** p95 158 ms to change a quantity and 127 ms to remove an item, against 76 ms to view the basket. Updates and deletes go to SQLite, which allows one writer at a time, so they would be the first to suffer at higher load.
- **Memory stayed flat** (279-286 MB) for the whole test, because each user empties their basket at the end of every pass.

Server CPU across the three modules

Juice Shop CPU during each load test (100% = one full core)



3-sample moving average. Ramp-up runs for the first 60 s, steady load to 240 s, then ramp-down.

At a similar 20-23 requests per second, each module keeps Juice Shop at **58-64% of one CPU core** under steady load. That is roughly 25-30 ms of CPU per request on average, whichever module is used. All three modules share the same single process, so they also share its limit. The stress test puts that limit at about 40-45 requests per second for search. Combined traffic across the whole site should hit the same ceiling at a similar total rate, so the three load tests running at once (about 63 req/s) would already exceed it. That last point is an estimate from these numbers, not a measured result.

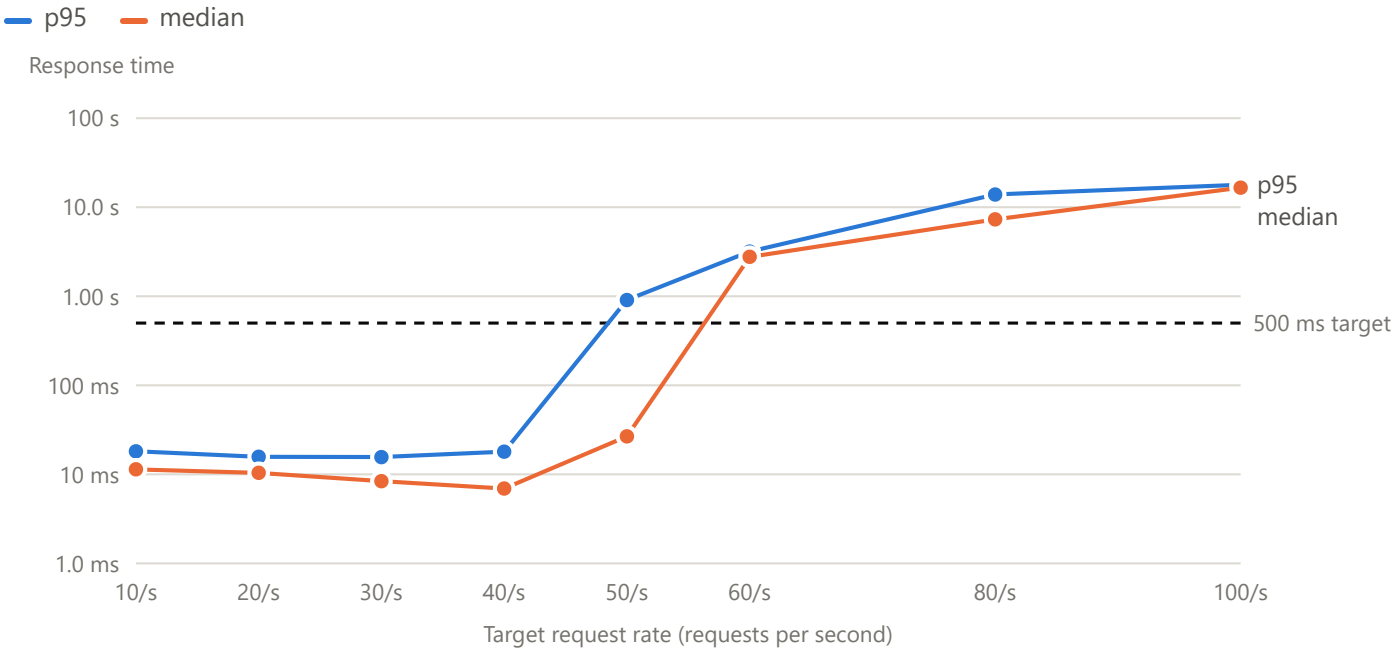
4. Stress test: finding the breaking point

The stress test sends product searches at a fixed rate that rises every minute: 10, 20, 30, 40, 50, 60, 80 and 100 requests per second. k6 keeps sending at the target rate even when responses slow down (an open model), so a slow server can't hide by slowing the test down. Requests time out after 30 s. Juice Shop was restarted before the run.

Target rate	Completed	Median	p95	Max	Errors	Result
10/s	9.6/s	11 ms	18 ms	34 ms	0.00%	✓ Pass
20/s	19.6/s	10 ms	16 ms	38 ms	0.00%	✓ Pass
30/s	29.6/s	8.4 ms	16 ms	60 ms	0.00%	✓ Pass
40/s	39.6/s	6.9 ms	18 ms	148 ms	0.00%	✓ Pass
50/s	49.6/s	27 ms	907 ms	1.17 s	0.00%	✗ Fail
60/s	58.0/s	2.77 s	3.17 s	3.27 s	0.00%	✗ Fail
80/s	68.5/s	7.31 s	13.9 s	14.4 s	0.00%	✗ Fail
100/s	61.0/s	16.6 s	17.8 s	18.0 s	0.00%	✗ Fail

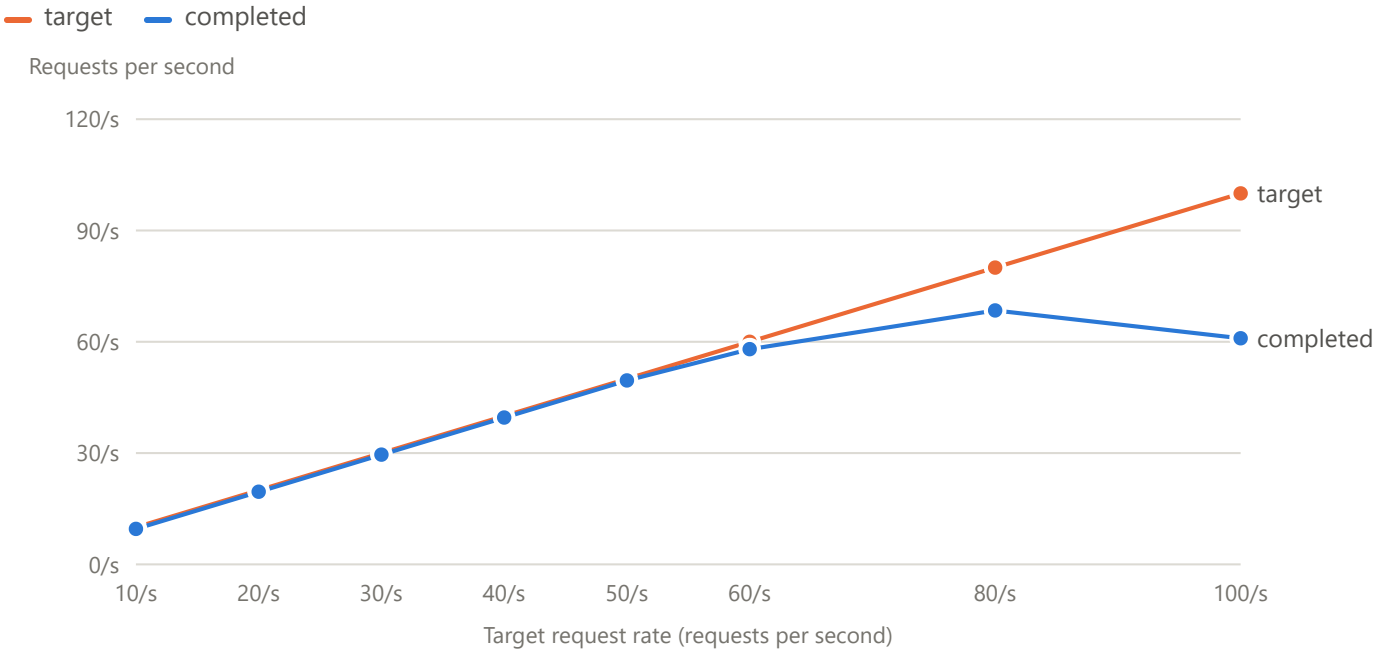
Dropped iterations (requests k6 could not start because every virtual user was waiting on a response): 3,030. A step passes when p95 < 500 ms and errors < 1%.

Stress test: search response time at each request rate



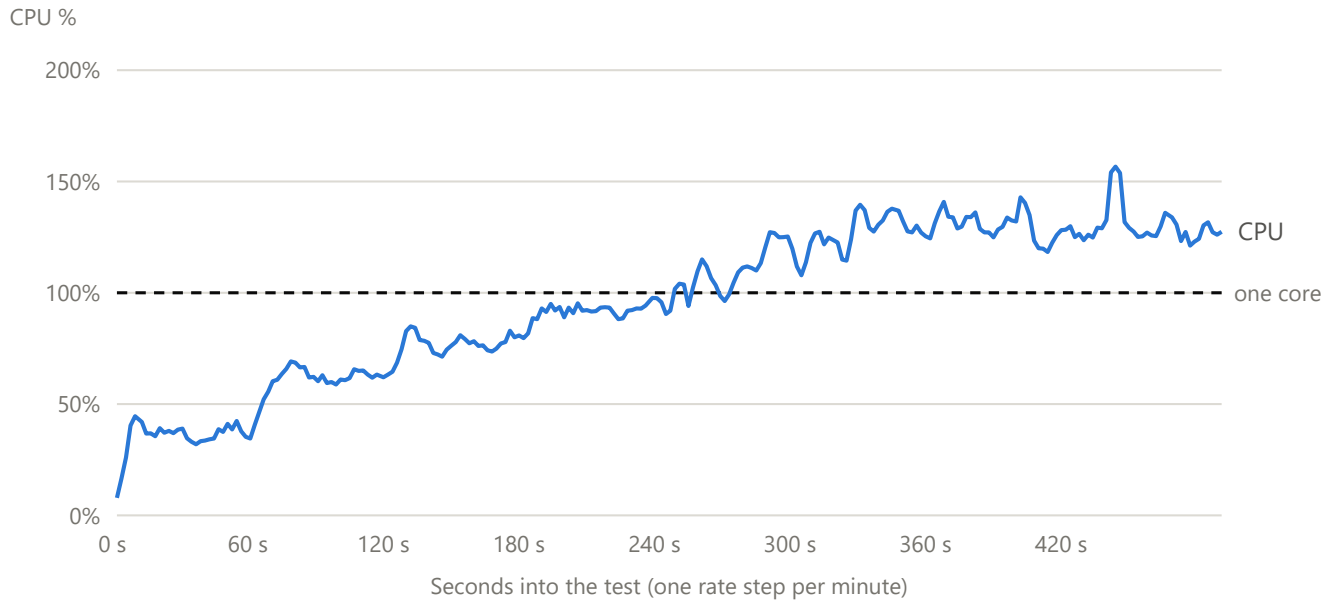
Log scale when values span several orders of magnitude.

Stress test: requests completed versus requests sent

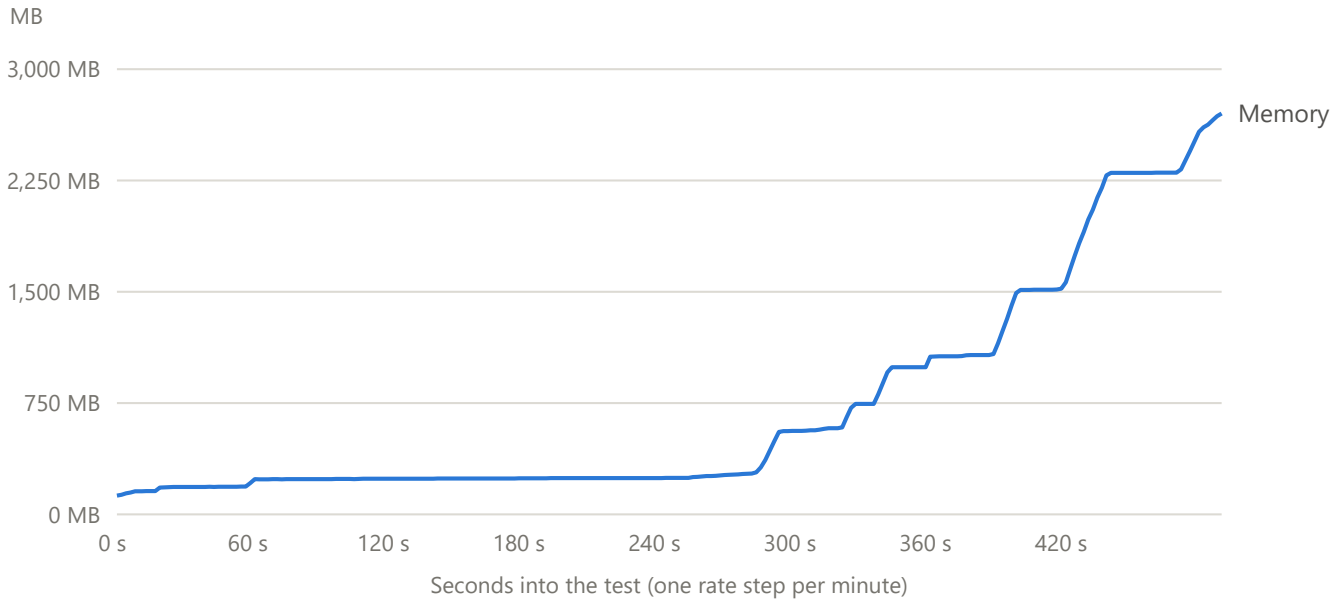


When the completed line falls below the target line, the server is no longer keeping up.

Stress test: Juice Shop CPU over time (100% = one full core)



Stress test: Juice Shop memory over time



Breaking point: between 40 and 50 requests per second.

- **10-40 req/s: healthy.** Every request completes, the median is 7-11 ms and p95 16-18 ms. Server CPU rises with the rate, from 36% to 92% of one core on average.
- **50 req/s: the knee.** CPU averages 109%, so the Node.js main thread is saturated (the extra comes from garbage collection and I/O threads). Requests start to queue: the median is still 27 ms, but p95 is 907 ms and fails the target.
- **60-100 req/s: overloaded.** Queues grow every second. p95 rises to 3.2 s, then 13.9 s, then 17.8 s. Throughput peaks at about 68 req/s and then *falls* to 61 req/s at a 100 req/s target, so more traffic gets less work done. k6 also had to drop 3,030 requests it couldn't start because all 1,000 virtual users were waiting on responses.
- **No errors, only waiting.** Every request eventually succeeded within the 30 s timeout. A real user would have given up long before.
- **Memory grows once requests queue.** It holds at about 245 MB up to 40 req/s, then climbs to 2.5 GB by the end of the test. Each queued request holds memory until it is answered.

So one Juice Shop process can serve about **40 searches per second within target**, roughly 23 ms of CPU per search (92% of a core at 40 req/s). That is low for an endpoint that returns at most 35 small rows. Most of the time is spent in the application's per-request work rather

than in the database.

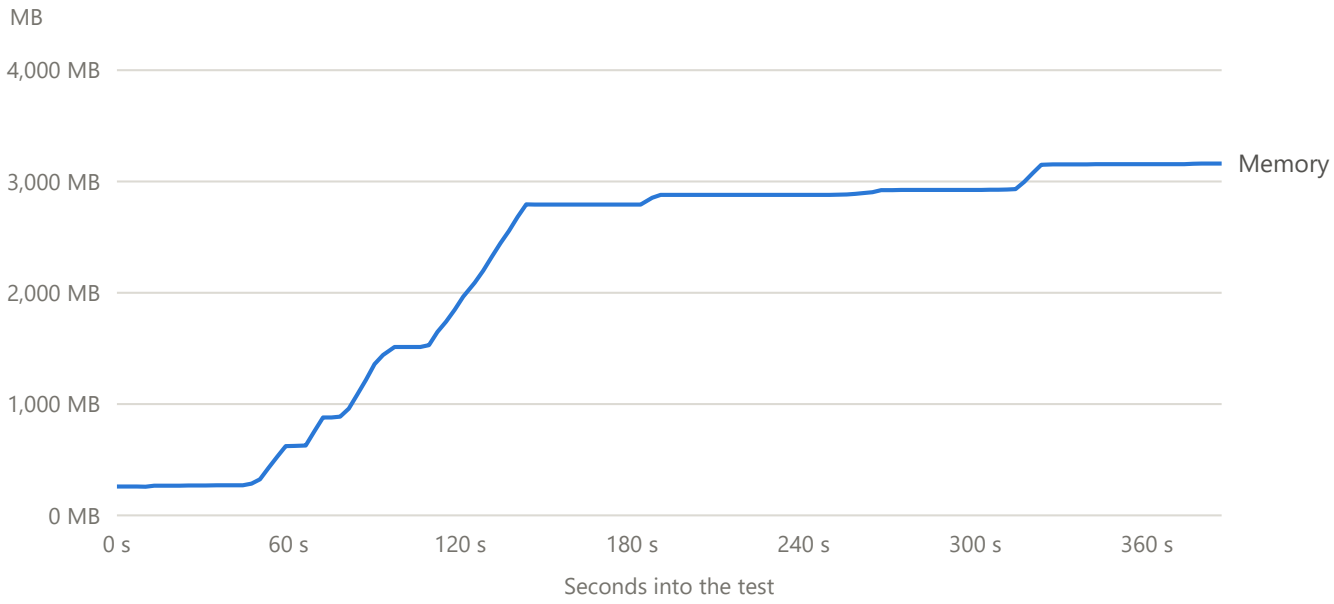
5. Overload run: what happens past the limit

The first stress run used much bigger steps (50, 100, 200, 400, 600 and 800 requests per second), before the breaking point was known. It overloaded the server from the first step, which is why the second run zooms in on 10-100 req/s. It is kept because it shows how Juice Shop behaves well past its limit.

Target rate	Completed	Median	p95	Errors
50/s	47.3/s	30 ms	2.56 s	0.00%
100/s	59.9/s	17.6 s	21.2 s	0.00%
200/s	47.6/s	21.7 s	30.0 s	13.30%
400/s	33.3/s	30.0 s	30.0 s	100.00%
600/s	33.3/s	30.0 s	30.0 s	100.00%
800/s	33.3/s	30.0 s	30.0 s	78.10%

Dropped iterations: 111,708. Requests time out after 30 s, which is why p95 sits at 30 s from 400/s.

Overload run: Juice Shop memory over time



- **Failure by timeout.** From 200 req/s, 13% of requests hit the 30 s timeout; from 400 req/s, all of them did. The server never returned an error itself. It queued work without limit until clients gave up.
- **Memory grew to 3.1 GB and stayed there.** It was still 3.0 GB with no traffic, after the test. A single search then took 128 ms, against about 45 ms on a fresh process. The second stress run showed the same pattern (2.8 GB after 30 s idle, search 81 ms).
- From this data alone we can't tell whether that memory is a leak or a heap that V8 doesn't hand back to the operating system. The slower responses afterwards suggest the large heap costs garbage-collection time either way. A heap snapshot would settle it (see Recommendations).

6. Bottlenecks and issues found

#	Finding	Evidence	Impact
1	**CPU-bound single process.** Juice Shop runs all requests on one Node.js main thread.	Stress: CPU reaches 100% of a core at about 45 req/s of search, exactly where latency breaks.	Capacity is about 40 searches/s per process, whatever the hardware has spare. The other 9 cores sat mostly idle.
2	**High CPU cost per request, in every module.**	About 23 ms of CPU per search at 40 req/s in the stress test, and 58-64% of a core at only 20-23 req/s in each load test (25-30 ms per request).	The cost is spread across all routes, so the whole site shares a ceiling of roughly 40 req/s per process.
3	**No load shedding.** Excess requests queue without limit instead of being rejected quickly.	Overload run: 0% server errors but 100% client timeouts from 400 req/s; throughput falls when overloaded.	One traffic spike makes the site unusable for everyone, rather than slow for a few.
4	**Memory retained after overload.**	Memory grows from about 250 MB to 2.5-3.1 GB during overload, stays there when traffic stops, and responses stay about 2x slower.	The app doesn't recover after a spike without a restart, and could run out of memory under a longer one.
5	**HTTP 500 on a duplicate basket item** (functional defect).	Adding a product that is already in the basket returns 500. Found while designing the basket test.	A server error for a normal user action. It also counts as an error in monitoring.
6	**No rate limit on failed logins** (security).	The login test made 497 wrong-password attempts (about 16 per account in 4.5 min), all answered in under 230 ms with no lockout.	Passwords can be guessed at full server speed. Login is also an obvious target for a request flood, given finding 1.

7. Recommendations

1. **Run several Node.js processes.** Use Node's `cluster` module or a process manager such as PM2, one process per core, behind a load balancer. Read-heavy endpoints like search should scale close to linearly. Writes to SQLite are serialised, so for production traffic move the database to PostgreSQL or MySQL as well.
2. **Profile and cache search.** Profile the search route with `node --cpu-prof` to find where the 23 ms of CPU goes. The catalogue is small and rarely changes, so cache search results in memory with a short expiry. That should remove most of the cost.
3. **Shed load instead of queueing.** Cap concurrent requests per process and answer the excess quickly with 503 and `Retry-After`, and set a server-side request timeout. Users then get a fast error instead of a 30 s wait, and memory stays bounded.
4. **Investigate the retained memory.** Take heap snapshots before, during and after an overload run (for example with `node --inspect` and Chrome DevTools) to see whether it is a leak. In the meantime, a health check that restarts a process whose memory stays high would contain the damage.
5. **Fix the duplicate basket item error.** Return 400 or 409, or increase the quantity of the existing item instead.

6. **Rate-limit failed logins** per account and per IP address (for example, slow down after 5 failures). This also protects the CPU budget from finding 1.
7. **Re-test on production-like infrastructure.** Put the load generator on a separate machine, run a soak test of an hour or more at about 30 searches per second to confirm memory stays flat below the limit, and repeat the stress test after each change above to measure the gain.

8. Limitations and workarounds

Limitation	Effect	Workaround or mitigation
k6 and Juice Shop on the same laptop	They compete for CPU. Under stress, k6 runs up to 1,000 virtual users, which takes CPU the app could have used.	The app is limited to about one core anyway (finding 1), and the laptop has 12 threads, so the effect on the breaking point should be small. For final numbers, run k6 from a separate machine.
Laptop CPU (i5-1235U, a low-power U-series chip)	Clock speed varies with temperature and power settings, so absolute numbers will differ on servers.	Compare results against each other on this machine, and treat absolute capacity as indicative.
Loopback network	No real network latency, bandwidth or TLS, so response times are server time only.	This isolates server performance, which is the goal. Add network latency when testing a deployed environment.
Juice Shop is a deliberately insecure training app	It isn't tuned for production (SQLite, a single process, challenge-tracking code on every request).	The findings describe this app as shipped. The recommendations explain what production deployment would change.
Short tests (4.5-8 min)	Slow leaks or degradation over hours wouldn't show up.	Recommendation 6: a soak test of an hour or more.
Server metrics are process-level, sampled every 2 s	They show CPU and the Windows working set, not the V8 heap or event-loop delay.	Enough to locate the bottleneck. Recommendations 2 and 4 cover deeper profiling.
The first run's CPU sampler interfered with the tests	It ran a system-wide CPU query every 2 s, which cost CPU itself and stretched its sampling interval to about 3 s, so its raw CPU figures were too high.	CPU was recalculated from the real sample times, the sampler was replaced with a lighter one built into `run.ps1`, and the three load tests were run again on a freshly started Juice Shop. The report uses the rerun's results. The difference was large: p95 for search fell from 154 ms to 50 ms, and for login from 326 ms to 88 ms. The first run is kept in `results/2026-10-27_12-47-01` as evidence.
Permission prompts blocked starting Juice Shop from the automation at first	The first runs needed Juice Shop started by hand.	`run.ps1` now starts and stops Juice Shop itself when it isn't running.

9. Reproducing these results

From a fresh clone on Windows, `.\run.ps1 all` downloads Node.js, Juice Shop and k6, starts Juice Shop, runs every test, and saves the results to `results\<run>\`. Each test can also be run alone, for example `.\run.ps1 stress`. Then `node docs\report\build-report.js` rebuilds this report from the newest results. See the README for details.

Smoke test before the runs: 100.00% of 8 checks passed, p95 42 ms. Generated by docs/report/build-report.js from these run folders: `product-search`: results/2026-10-27_13-20-07, `login`: results/2026-10-27_13-20-07, `basket`: results/2026-10-27_13-20-07, `stress`: results/2026-10-27_13-10-05, `stress-overload`: results/2026-10-27_12-47-01, `smoke`: results/2026-10-27_12-47-01.